

PYTHON DAY 5

Strings, Text Processing, Indexing, Slicing, and Formatting

Total time: 3 to 4 hours | **Level:** Beginner | **Main goal:** Understand text as data and learn to inspect, transform, search, and format strings.

Today's Big Idea

A string is not just something you print. It is a sequence of characters that your programs can inspect and transform. Today you learn indexing, slicing, string methods, immutability, searching, splitting, joining, and f-strings.

```
Text
↓
String object
↓
Index / slice / search
↓
Transform
↓
Format
↓
Useful program
```

1. Warm-up From Memory - 15 minutes

Create warmup.py without opening Day 4. Ask for 5 numbers using a loop and print their sum. Then print how many numbers were entered. Do not use a list yet.

2. Creating Strings - 20 minutes

```
name = "Python"
message = 'Hello world'
paragraph = """This is
multiple lines."""

print(name)
print(message)
print(paragraph)
```

Experiment with quotes inside strings and newline characters such as `\n` and tabs such as `\t`. Research: does Python treat single-quoted and double-quoted strings differently?

3. Strings Are Sequences - 35 minutes

```
text = "PYTHON"

print(text[0])
print(text[1])
print(text[-1])
print(text[-2])
```

Draw the indexes on paper:

```
P Y T H O N
0 1 2 3 4 5
-6 -5 -4 -3 -2 -1
```

Research: why does indexing start at 0? Why is negative indexing useful?

4. Slicing - 35 minutes

```
text = "PYTHON"

print(text[0:2])
print(text[1:4])
```

```
print(text[:3])
print(text[3:])
print(text[:])
print(text[:2])
print(text[::-1])
```

Predict every result before running. Focus on the rule: start is included, stop is excluded. Challenge: reverse a string without a loop.

Official reading: <https://docs.python.org/3/tutorial/introduction.html#strings>

5. String Immutability - 25 minutes

```
text = "Python"
text[0] = "J"
```

Read the error carefully. Then create a new string:

```
text = "Python"
text = "J" + text[1:]
print(text)
```

Research: what does immutable mean? Why can a variable be rebound even though the string object itself cannot be changed?

6. Common String Methods - 40 minutes

```
text = " Hello Python World "

print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("Python", "Programming"))
print(text.find("Python"))
print(text.count("o"))
print(text.startswith(" Hello"))
print(text.endswith(" "))
```

Experiment:

```
text = "python"
result = text.upper()
print(text)
print(result)
```

Notice that many string methods return a new string. Research 5 additional string methods and document their purpose, arguments, and return value.

Official reference: <https://docs.python.org/3/library/stdtypes.html#string-methods>

7. Searching and Membership - 25 minutes

```
text = "Python is powerful"

print("Python" in text)
print("Java" in text)
print("python" in text)
print("Python" not in text)
```

Explain why the third expression is False. Then use membership checks inside if statements.

8. split() and join() - 35 minutes

```
sentence = "Python is fun to learn"
words = sentence.split()
print(words)

joined = "-".join(words)
print(joined)
```

Experiment with comma-separated data:

```
data = "apple, banana, orange"
items = data.split(",")
print(items)
```

Research why split() and join() are complementary operations.

9. f-Strings - 30 minutes

```
name = "Preetham"
age = 28
city = "Hyderabad"

print(f"My name is {name}.")
print(f"I am {age} years old and live in {city}.")
```

Expressions can go inside braces:

```
price = 1250
quantity = 3
print(f"Total: {price * quantity}")
```

Try number formatting:

```
price = 1250.5678
print(f"Price: {price:.2f}")
```

Research: why are f-strings often easier to read than manual string concatenation?

10. Mini Project - Username Cleaner - 30 minutes

Create `username_cleaner.py`. Ask for a messy name and produce a clean username. Use `strip()`, `lower()` or `upper()`, `replace()`, and at least one membership check. Test at least 10 inputs, including empty input and extra spaces.

11. Practical Project - Text Statistics Analyzer - 45 minutes

Create `text_analyzer.py`. Ask for a sentence or paragraph and report character count, word count, occurrences of a chosen word or character, first and last character when non-empty, keyword membership, uppercase/lowercase versions, and a clean f-string report.

12. Research Experiment - `len()`, Unicode, and Characters - 25 minutes

```
text = "Python"
print(len(text))

text = "café"
print(len(text))

text = "■"
print(len(text))
```

Research at a beginner level: what is Unicode, and why can a visible character involve more complexity than one byte? The goal is curiosity, not memorization.

13. Debugging Lab - 25 minutes

```
name = input("Name: ")
print(name.upper)
```

```
text = "Python"
print(text[10])
```

```
text = "Python"
print(text[1:100])
```

```
text = "hello"
text.upper()
print(text)
```

For each example, determine whether it is an error or simply code that does not do what the programmer intended. Explain the reason and fix it.

14. Main Project - Log / Message Analyzer - 60 minutes

Build `message_analyzer.py`. Put several sample application log messages into a multi-line string. Analyze the text and produce a report.

Requirements:

- Count total characters and words.
- Count occurrences of ERROR and WARNING.
- Check whether a specific service name appears.
- Normalize case before searching.
- Use `splitlines()` to process individual log lines with a for loop.
- Print a clean summary with f-strings.

Bonus: make searching case-insensitive and count errors per line.

15. Knowledge Check

- Q1. What is a string in Python?
- Q2. Why does indexing start at 0?
- Q3. What does `text[-1]` return?
- Q4. What does `text[1:4]` include and exclude?
- Q5. What does immutable mean for strings?
- Q6. Why does `text.upper()` not automatically change `text`?
- Q7. What does `strip()` do?
- Q8. What is the difference between `find()` and `in` at a high level?
- Q9. What does `split()` return?
- Q10. What does `join()` do?
- Q11. Why are f-strings useful?
- Q12. What is Unicode at a high level?

16. Day 5 Completion Checklist

- [] I can create and inspect strings.
- [] I understand positive and negative indexing.

- [] I can slice strings.
- [] I understand basic string immutability.
- [] I can use common string methods.
- [] I can use in and not in.
- [] I can split and join text.
- [] I can use f-strings.
- [] I can debug common string problems.
- [] I completed Username Cleaner.
- [] I completed Text Statistics Analyzer.
- [] I completed Log / Message Analyzer.
- [] I tested empty input, long input, mixed case, and unexpected characters.

Recommended Study Schedule

15 min warm-up | 20 min creation | 35 min indexing | 35 min slicing | 25 min immutability | 40 min methods | 25 min searching | 35 min split/join | 30 min f-strings | 30 min mini project | 45 min text analyzer | 25 min Unicode research | 25 min debugging | 60 min main project.

Official References

Python Tutorial, Strings: <https://docs.python.org/3/tutorial/introduction.html#strings>

String Methods: <https://docs.python.org/3/library/stdtypes.html#string-methods>

Built-in Functions: <https://docs.python.org/3/library/functions.html>

Day 6 Preview

Next: lists, indexing, mutation, append, insert, remove, pop, sorting, copying, iteration, and practical data-processing patterns.